PrimeX AI

A Modular, Vendor-Independent Personal AI Operating System

Production-Grade Architecture on Zero-Cost Infrastructure

Complete Project Specification & Engineering Submission

Prepared By	PrimeX Engineering (Solo Developer / Architect of Record)
Document Type	Final Project Specification
Version	1.0
Date	25 June 2026
Document Status	FINAL — Scope Frozen
Classification	Confidential — For Review Only

Confidentiality Statement

This document contains the complete architecture and engineering record of the PrimeX AI project. It is prepared for academic, internship, technical-review-board, and engineering-management evaluation. It describes a single-user, non-commercial personal system and contains no proprietary third-party material.

Table of Contents

Table of Contents....2
Executive Summary....6
Project Overview....6
Problem Statement....6
Solution Overview....6
Business and Portfolio Value....7
Expected Outcomes....7
Chapter 1 — Introduction....8
1.1 Background....8
1.2 Existing Problems....8
1.3 Need for the System....8
1.4 Objectives....8
1.5 Scope....9
1.6 Benefits....9
1.7 Target Users....9
Chapter 2 — Project Overview....11
2.1 Product Vision....11
2.2 Core Features....11
2.3 User Roles....11
2.4 User Workflows....12
2.5 Business Requirements....12
2.6 Functional Requirements....12
2.7 Non-Functional Requirements....13
Chapter 3 — System Analysis....14
3.1 Requirement Analysis....14
3.2 Feasibility Study....14
3.3 Risk Analysis....15
3.4 Assumptions....15
3.5 Constraints....15
Chapter 4 — System Architecture....16
4.1 High-Level Architecture....16
4.2 Component Architecture....16
4.3 Service Architecture....16
4.4 AI Architecture....17
4.5 Data Flow....17
4.6 Security Architecture....17
4.7 Scalability Strategy....18
4.8 Reliability Strategy....18
4.9 Monitoring Strategy....18
Chapter 5 — Database Design....19
5.1 Database Overview....19
5.2 Entity Relationship Design....19
5.3 Table Structures....19
5.4 Relationships....20
5.5 Indexing Strategy....20

5.6 Data Retention Strategy....20
5.7 Backup Strategy....21
Chapter 6 — AI & Intelligence Layer....22
6.1 AI Goals....22
6.2 Model Architecture....22
6.3 Routing Layer....22
6.4 RAG Pipeline....22
6.5 Memory System....23
6.6 Prompt Engineering Framework....23
6.7 Context Management....23
6.8 Safety Layer....23
6.9 Future AI Enhancements....24
Chapter 7 — Security Strategy....25
7.1 Authentication....25
7.2 Authorization....25
7.3 Session Management....25
7.4 Encryption....25
7.5 API Security....26
7.6 Rate Limiting....26
7.7 Audit Logging....26
7.8 Compliance Considerations....26
Chapter 8 — Implementation Details....27
8.1 Frontend Modules....27
8.2 Backend Modules....27
8.3 API Layer....27
8.4 Storage Layer....27
8.5 Background Jobs....28
8.6 Event Processing....28
8.7 Deployment Pipeline....28
Chapter 9 — Testing Strategy....29
9.1 Unit Testing....29
9.2 Integration Testing....29
9.3 End-to-End Testing....29
9.4 Security Testing....29
9.5 Performance Testing....29
9.6 Load Testing....30
9.7 User Acceptance Testing....30
Chapter 10 — Deployment Architecture....31
10.1 Development Environment....31
10.2 Staging Environment....31
10.3 Production Environment....31
10.4 CI/CD Pipeline....31
10.5 Infrastructure Layout....31
10.6 Monitoring & Alerting....32
10.7 Disaster Recovery....32
Chapter 11 — Project Roadmap....33

Phase 1 — Foundation: Authentication, Chat, Gateway, and Storage Foundations..33
Phase 2 — Document Intelligence....33
Phase 3 — Knowledge Bases and RAG....34
Phase 4 — Memory System....34
Phase 5 — Provider Routing....35
Phase 6 — Search and Administration....35
Phase 7 — Advanced File Intelligence....35
Phase 8 — Voice and Agents....36
Phase 9 (Roadmap) — Personal Knowledge Graph....36
Chapter 12 — Engineering Decisions....37
Decision 12.1 — Storage Inversion: Neon as Index, R2 as System of Record....37
Decision 12.2 — Cross-Origin Authentication Topology....37
Decision 12.3 — Half-Precision Embeddings at 768 Dimensions....38
Decision 12.4 — GitHub Actions for Scheduled and Heavy Background Work....38
Decision 12.5 — Vendor-Independent AI Gateway with Fallback Chain....39
Decision 12.6 — Self-Hosted Stateless Authentication....39
Decision 12.7 — API-Based Embeddings over Local Models....39
Decision 12.8 — Hard-Delete Archival over Soft Deletion....40
Decision 12.9 — pgvector over a Dedicated Vector Database....40
Decision 12.10 — Knowledge Graph Deferred to a Roadmap Phase....41
Chapter 13 — Risks & Mitigation....42
13.1 — Active-Database Storage Exhaustion....42
13.2 — Primary Provider Free-Tier Volatility....42
13.3 — Loss of In-Process Background Tasks....42
13.4 — Silent Retrieval-Quality Failure....43
13.5 — Memory Degradation....43
13.6 — Agent Quota Self-Denial-of-Service....43
13.7 — Cross-Store Consistency Orphans....44
13.8 — Prompt Injection via Retrieved Content....44
13.9 — Provider Model Deprecation and Dependency Drift....44
13.10 — Cold-Start Latency Perceived as Failure....45
Chapter 14 — Project Management....46
14.1 Team Structure....46
14.2 Roles & Responsibilities....46
14.3 Development Workflow....46
14.4 Release Workflow....47
14.5 Documentation Strategy....47
14.6 Quality Assurance Process....47
Chapter 15 — Future Enhancements....48
Short-Term Enhancements....48
Mid-Term Enhancements....48
Long-Term Vision....48
Chapter 16 — Conclusion....49
Project Summary....49
Achievements....49
Expected Impact....49

Final Remarks....49
Appendix A — Complete Folder Structure....50
Appendix B — API Catalog....52
Appendix C — Database Catalog....54
C.1 Detailed Column Schemas....54
Appendix D — Security Checklist....60
Appendix E — Deployment Checklist....61
Appendix F — Testing Checklist....62
Appendix G — Glossary....63
Appendix H — References....64

Executive Summary

Project Overview

PrimeX AI is a personal Artificial Intelligence Operating System: a single-user platform that unifies conversational AI, document intelligence, retrieval-augmented generation (RAG), long-term memory, multi-provider model routing, search, and autonomous agents into one coherent, modular system. It is built deliberately as a long-term personal ecosystem rather than a minimum-viable product, and it is engineered to run entirely on free-tier infrastructure at a recurring cost of zero dollars per month.

The defining characteristic of PrimeX AI is not any single feature, but the discipline with which a production-grade architecture is mapped onto severely constrained free infrastructure. The system is designed so that the same architecture which runs on a 0.5 GB database and a 512 MB application instance today could scale to enterprise infrastructure tomorrow without a redesign.

Problem Statement

Modern knowledge workers accumulate information across dozens of disconnected tools: chat assistants forget prior conversations, documents are siloed in folders, and personal knowledge is never consolidated into a queryable, evolving form. Commercial AI assistants are powerful but stateless with respect to the individual user's complete corpus, and they do not give the user ownership of the underlying memory, retrieval, or routing logic. For an engineer seeking to learn modern AI systems engineering, there is additionally no single project that exercises authentication, data engineering, RAG, vector search, memory systems, multi-provider orchestration, and autonomous agents end-to-end under real operational constraints.

Solution Overview

PrimeX AI addresses both problems with one design. A FastAPI backend owns all business logic and exposes an API-first contract; a Next.js frontend consumes it. All model calls pass through a single AI Gateway that abstracts Gemini, Groq, and OpenRouter behind a uniform interface with automatic fallback and health-aware routing. A three-layer storage model treats Cloudflare R2 as the durable system of record and Neon PostgreSQL as a bounded, fast working set, with GitHub Actions performing all scheduled archival, rollup, and backup work. Retrieval is grounded in pgvector using compact half-precision embeddings, and a curated memory subsystem provides persistent, deduplicated, contradiction-aware context across sessions.

Business and Portfolio Value

As a non-commercial project, PrimeX AI's value is measured in learning and demonstrable engineering capability rather than revenue. It demonstrates, in a single deployable artifact, the competencies most sought in 2026 AI engineering roles: RAG with citations, vector database operation, multi-provider reliability engineering, secure stateless authentication, data-lifecycle engineering under hard limits, and safe agent design. The constraint engineering — making a sophisticated system genuinely free to operate for years — is itself a senior-level signal that distinguishes the project from typical portfolio chatbots.

Expected Outcomes

- A fully deployed, single-user AI Operating System spanning eight delivered phases, operating at zero recurring cost.
- A storage lifecycle proven sustainable for multiple years within a 0.5 GB active database and a 10 GB object store.
- A reusable, vendor-independent AI Gateway that can adopt or drop providers through configuration alone.
- A complete, professional engineering record — this document — suitable for academic and professional review.

Chapter 1 — Introduction

1.1 Background

The period from 2023 to 2026 saw large language models move from novelty to infrastructure. Conversational assistants, retrieval-augmented generation, and autonomous agents became standard building blocks, while the surrounding engineering — provider orchestration, vector storage, memory, and cost control — remained the genuinely difficult and differentiating work. PrimeX AI was conceived as a vehicle to master that surrounding engineering by building, not merely studying, a complete system.

The project is explicitly framed as a long-term personal ecosystem. It is not a hackathon entry, a proof of concept, or a startup MVP. Its development philosophy is to build once, build correctly, scale gradually, minimise rewrites, and protect architectural consistency across the entire lifecycle.

1.2 Existing Problems

- Commercial assistants do not persist or expose the user's complete personal corpus as a queryable, evolving structure.
- Personal knowledge is fragmented across documents, chats, and notes with no unifying retrieval layer.
- Reliance on a single AI provider creates fragility: quota cuts, outages, and model deprecations break dependent applications.
- For a learner, no single off-the-shelf project exercises the full breadth of modern AI systems engineering under real constraints.

1.3 Need for the System

A unified personal AI platform is needed that (a) consolidates documents, conversations, and memory into one retrievable knowledge base; (b) is independent of any single model vendor; (c) preserves the original source material as the system of record so that derived artefacts such as embeddings can always be regenerated; and (d) operates indefinitely at no cost for a single user. PrimeX AI is the realisation of that need.

1.4 Objectives

- Deliver a modular, production-grade personal AI Operating System across eight phases.

- Maintain strict architectural consistency and avoid future rewrites through generous, forward-compatible schema design.
- Operate entirely on free-tier infrastructure at a recurring cost of zero dollars per month, for a single user, indefinitely.
- Demonstrate, in one artefact, authentication, the AI Gateway, RAG, memory, provider routing, search, advanced document intelligence, voice, and agents.
- Engineer a storage lifecycle that remains sustainable for multiple years within free-tier database and object-storage limits.

1.5 Scope

In scope: a single-user system delivering all eight approved phases — Foundation (auth, chat, gateway), Document Intelligence, Knowledge Bases and RAG, Memory, Provider Routing, Search and Administration, Advanced File Intelligence, and Voice and Agents — deployed across GitHub, Vercel, Render, Neon, and Cloudflare R2.

Out of scope (permanently, by design): multi-tenancy, commercial launch, paying customers, custom domains, paid services of any kind, and heavyweight infrastructure such as Kubernetes, Docker Swarm, microservices, Redis, Celery, Kafka, RabbitMQ, or Elasticsearch. A Personal Knowledge Graph is scoped as a post-Phase-8 (Phase 9) roadmap item rather than a committed phase.

1.6 Benefits

- A genuinely useful personal knowledge and conversation platform owned and operated entirely by its single user.
- Vendor independence: model providers can be added, removed, or reprioritised without code changes.
- Durable data ownership: originals and extracted text are preserved in object storage as the system of record.
- Demonstrable, resume-grade evidence of full-stack and AI systems engineering competence.

1.7 Target Users

PrimeX AI has exactly one user for its entire lifecycle: the developer-operator who builds, owns, and uses it. This single-user assumption is non-negotiable and is exploited throughout the architecture to eliminate the complexity of multi-tenancy, while a clean

user-ownership boundary (a user identifier on every owned record) preserves the option of future generalisation without committing to it now.

Chapter 2 — Project Overview

2.1 Product Vision

To build a modular, production-grade, vendor-independent AI Operating System that scales from free-tier infrastructure to enterprise architecture without redesign. PrimeX AI is intended to be intelligent in conversation, grounded in the user's own documents, persistent in memory, independent of any single provider, secure by default, and cost-optimised to the point of being free to operate.

2.2 Core Features

The product is delivered as eight phases, each shipping usable capability while preparing the ground for the next:

Phase	Capabilities
Phase 1	Authentication, user profile, streaming chat, chat history, the AI Gateway, Gemini primary with Groq fallback, usage tracking, monitoring foundation, and the storage and automation foundations.
Phase 2	Document upload (PDF, DOCX, TXT), original and extracted-text storage in R2, summarisation, single-document question answering.
Phase 3	Knowledge bases (collections), RAG, chunking, embedding, pgvector semantic search, citations, and the vector-archival lifecycle.
Phase 4	Memory system: short-term and long-term memory, preferences, facts, goals, deduplication, contradiction handling, scored retrieval.
Phase 5	Multi-provider routing with OpenRouter, provider health and cooldown states, and hardened fallback logic.
Phase 6	Search engine with citations, search history, and an administrative dashboard with usage, storage, and health analytics.
Phase 7	Advanced file intelligence: ZIP, CSV, XLSX, and PPTX analysis through the existing pipeline.
Phase 8	Voice features (browser-based), autonomous agents calling the system's own tools, and workflow automation.

2.3 User Roles

The system defines two nominal roles, both held by the same single person: a standard user role for all primary functionality, and an administrator role for the operational dashboard introduced in Phase 6. Role-based access control is implemented as a single

field on the user record; it is intentionally minimal because the system is single-user, while remaining present so that the authorization seam exists if ever needed.

2.4 User Workflows

Conversational workflow: The user opens a chat, sends a message, and receives a streamed response routed through the AI Gateway; the conversation is persisted and retrievable, with relevant long-term memories injected as context.

Document workflow: The user uploads a document; the original and its extracted text are stored in R2; the user requests a summary or asks grounded questions answered from the document.

Knowledge workflow: The user groups documents into a collection; the system chunks and embeds them; the user queries the collection and receives a cited, grounded answer, or an explicit abstention when retrieval is weak.

Memory workflow: The user records or confirms durable facts, preferences, and goals; the system deduplicates and scores them and recalls them in future sessions.

Agentic workflow: The user invokes an agent that performs a multi-step task by calling the system's own tools under strict iteration and quota budgets.

2.5 Business Requirements

- The system shall operate at zero recurring monetary cost for a single user, indefinitely.
- The system shall deliver all eight approved phases without feature removal.
- The system shall remain maintainable by a single developer with modest ongoing effort.
- The system shall constitute a credible, resume-grade demonstration of AI systems engineering.

2.6 Functional Requirements

Area	Requirement
Authentication	Register, log in, refresh session, log out, and retrieve the current user profile securely.
Chat	Create and manage multiple chat sessions with streaming responses and persistent, paginated history.
AI Gateway	Route all model calls through one abstraction with automatic fallback,

Area	Requirement
	usage accounting, and health awareness.
Documents	Upload, store, extract, summarise, and answer questions over PDF, DOCX, and TXT files.
RAG	Chunk, embed, retrieve, assemble context, and generate cited answers grounded in collections.
Memory	Create, read, update, delete, deduplicate, and retrieve long-term memories with scoring.
Routing	Select among Gemini, Groq, and OpenRouter using health and cooldown state.
Search	Perform web search with resolvable citations and persist search history.
Advanced files	Process ZIP, CSV, XLSX, and PPTX through the existing document pipeline.
Voice & Agents	Provide browser voice input/output and execute bounded, tool-using agents.

2.7 Non-Functional Requirements

Attribute	Requirement
Cost	Zero dollars per month, perpetually, within published free-tier limits.
Performance	Interactive responses stream incrementally; cold starts are tolerated as an accepted free-tier characteristic.
Storage	The active database working set must remain within 0.5 GB indefinitely through archival.
Reliability	Provider failure must degrade gracefully via fallback rather than failing the request.
Security	Stateless JWT authentication, hashed secrets, transport encryption, and strict input validation throughout.
Maintainability	Modular, loosely coupled components, each independently replaceable; generous schema, lazy code.
Extensibility	New providers, file types, and memory behaviours can be added without architectural change.
Observability	Every model request is logged with provider, model, latency, tokens, and status; errors flow to Sentry.

Chapter 3 — System Analysis

3.1 Requirement Analysis

Requirements were derived top-down from the product vision and bottom-up from the constraints of free-tier infrastructure. The central analytical insight is that the binding constraint is not compute or bandwidth but the 0.5 GB active-database limit; almost every functional requirement that writes data must therefore be analysed for its storage-growth behaviour, and the architecture must provide an archival path for any unbounded data set. A second analytical insight is that provider quotas are volatile and have been cut without notice in the recent past, so multi-provider fallback is a core requirement rather than a later enhancement.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The project is technically feasible. Every component maps to a mature, well-documented technology with a genuine free tier: Next.js and FastAPI for the application, PostgreSQL with pgvector for relational and vector storage, Cloudflare R2 for object storage, and Gemini, Groq, and OpenRouter for inference. The principal technical risks — cross-origin authentication, streaming, the absence of a free background-worker, and silent retrieval-quality failure — are each addressed by a specific, proven design decision documented in Chapter 12.

3.2.2 Operational Feasibility

The project is operationally feasible for a single developer working five to six hours per day over five to six months. Operational burden after launch is concentrated in the storage lifecycle (archival, rollups, backups) and in periodic maintenance (model repinning, dependency updates), all of which are automated where possible through GitHub Actions and bounded by design.

3.2.3 Economic Feasibility

The project is economically feasible at a target of zero recurring cost. All infrastructure, inference, monitoring, and tooling are consumed within free tiers sized far above single-user demand on every axis except active-database storage, which the storage lifecycle explicitly defends. The only monetary cost associated with the project is the developer's existing AI assistant subscription, which is excluded from the operating budget by assumption.

3.2.4 Security Feasibility

The project is security-feasible using standard, well-understood mechanisms: stateless JWT access tokens, rotating hashed refresh tokens, bcrypt password hashing, transport-layer encryption provided by the hosting platforms, strict server-side input and file validation, and per-user rate limiting implemented in the database. The single-user model substantially reduces the attack surface relative to a multi-tenant system.

3.3 Risk Analysis

Risks are catalogued in full in Chapter 13. In summary, the highest-severity risks are storage exhaustion of the active database, volatility of the primary model provider's free tier, loss of in-process background tasks on a sleeping instance, and silent degradation of retrieval and memory quality over time. Each is paired with a specific mitigation that is built into the architecture from the first phase rather than retrofitted.

3.4 Assumptions

- The system will remain single-user for its entire lifecycle.
- Free-tier limits for the chosen platforms remain broadly viable for single-user volumes, with the understood risk that providers may adjust them.
- The extracted text of every document is the system of record; embeddings and other derived artefacts are disposable and regenerable.
- The developer accepts that free-tier inference may use submitted prompts to improve the provider's models, a privacy trade-off inherent to operating at zero cost.

3.5 Constraints

The following constraints are non-negotiable and shape every decision in this document:

- Single-user; no multi-tenancy; no commercial launch; no custom domain.
- Deployment only on GitHub, Vercel Free, Render Free, Neon Free, and Cloudflare R2 Free; inference only on Gemini, Groq, and OpenRouter free tiers.
- Budget of zero dollars per month; timeline of five to six months; daily effort of five to six hours.
- No feature removals; no heavyweight infrastructure (Kubernetes, Docker Swarm, microservices, Redis, Celery, Kafka, RabbitMQ, Elasticsearch).

Chapter 4 — System Architecture

4.1 High-Level Architecture

PrimeX AI is organised into three layers plus an external provider tier. The Client Layer (Next.js on Vercel) renders the interface and consumes the backend API. The Application Layer (FastAPI on Render) owns all business logic, including the AI Gateway, the RAG orchestrator, the memory engine, and the storage adapters. The Data Layer comprises Neon PostgreSQL with pgvector as the bounded active working set and Cloudflare R2 as the durable system of record. External to these layers sit the AI Providers (Gemini, Groq, OpenRouter), reached only through the AI Gateway, and Sentry, which receives error and performance signals from both the client and the application.

A strict rule governs the layers: the frontend never calls an AI provider directly, and no module other than the AI Gateway imports a provider SDK. PostgreSQL is the source of truth for relational state; extracted text in R2 is the source of truth for content; embeddings are disposable derivations of that text.

4.2 Component Architecture

Within the Application Layer, components are modular and loosely coupled, each replaceable in isolation. The principal components are the Authentication service, the Chat service, the AI Gateway with its provider adapters, the Document service and extraction pipeline, the RAG orchestrator (chunker, embedder, retriever, reranker), the Memory engine, the Search service, the Agent runtime, the R2 storage adapter, and the monitoring and usage-tracking utilities. Components communicate through service interfaces and thin repositories rather than shared mutable state, which is what permits any one of them to be extracted or rewritten without disturbing the others.

4.3 Service Architecture

The backend follows a layered service architecture: API routers handle transport and validation; service classes hold business rules and orchestration; repositories encapsulate data access; and models define persistence. This separation keeps the API thin, the business logic testable in isolation, and the data layer swappable. Background work is expressed as plain task functions taking scalar arguments, so that the execution mechanism — currently GitHub Actions and in-process tasks — can change later without rewriting the logic.

4.4 AI Architecture

All intelligence flows through the AI Gateway, which exposes a uniform interface for chat generation and embedding and hides provider-specific details. The Gateway selects a provider according to configured priority and live health, dispatches the request, records usage, and on a classified retryable failure falls through to the next provider. Embeddings are always produced by a single configured model so that the vector space remains consistent; a dimension check guarantees that every stored vector matches the platform standard of 768 dimensions. The Gateway is the seam that makes the entire system vendor-independent: adopting or dropping a provider is a configuration and adapter change, never a change to calling code.

4.5 Data Flow

4.5.1 Chat Flow

A user message arrives at the chat API, which loads the recent conversation window and any relevant long-term memories, assembles a prompt, and calls the Gateway for a streamed completion. Tokens stream to the browser via Server-Sent Events directly from the application instance. The completed message is persisted; if the stream fails after it begins, partial content is preserved and marked incomplete, and the user is offered a retry rather than a silent provider switch.

4.5.2 RAG Flow

A document is uploaded and its original stored in R2; text is extracted and also stored in R2; the text is chunked; each chunk is embedded through the Gateway and stored as a half-precision vector in pgvector together with a pointer to its full text in R2. At query time the question is embedded with the same model, the top matching chunks are retrieved, their full text is fetched from R2, and a grounded, cited answer is generated. When retrieval similarity falls below a configured floor, the system abstains rather than fabricating an answer.

4.6 Security Architecture

Authentication is stateless and token-based. A short-lived JWT access token, held only in browser memory, authorises every API call via a bearer header; a long-lived opaque refresh token, stored hashed in the database and delivered as a Secure, HttpOnly cookie scoped to the refresh endpoint, is used solely to mint new access tokens. Because the frontend and backend are served from different origins, the refresh cookie is configured SameSite=None with Secure, and all other calls rely on the cross-origin-safe bearer

header. Refresh tokens rotate on use and are protected against the concurrent-refresh race by single-flight handling on the client and a short grace window on the server.

4.7 Scalability Strategy

Scalability for a single user is reframed as sustainability: the system must run for years without exceeding free limits, not serve many concurrent users. The active database is kept bounded by an archival engine that evicts cold data to R2; the chat history is a rolling window; high-velocity operational tables are rolled up to daily aggregates. Because R2 is effectively unlimited at single-user scale and charges nothing for egress, the corpus may grow without bound there while the queryable working set in Neon oscillates within its budget. The same component boundaries that enable this would also permit horizontal extraction onto paid infrastructure if the single-user assumption were ever lifted.

4.8 Reliability Strategy

Reliability rests on graceful degradation. The AI Gateway's fallback chain ensures that a single provider's outage or quota exhaustion does not fail a request. Provider health is tracked with cooldown states so that a failing provider is skipped until it recovers. Background work is made idempotent and re-triggerable so that a task lost to an instance restart can be safely replayed. Weekly database backups to R2, with a tested restore procedure, protect against data loss; cross-store consistency between R2 and Neon is reconciled by a scheduled sweep that detects and resolves orphans.

4.9 Monitoring Strategy

Every model request is recorded with timestamp, provider, model, latency, token counts, status, and error type. System-level signals — storage usage, active vector count, collection counts, file-processing status, and provider health — are surfaced in the Phase 6 administrative dashboard, which reads from the existing usage and health tables rather than introducing a separate analytics system. Application exceptions and performance traces flow to Sentry. A storage alert fires when the active database crosses eighty per cent of its budget, triggering defensive archival well before any hard limit is reached.

Chapter 5 — Database Design

5.1 Database Overview

The database is Neon PostgreSQL with the pgvector extension. Its governing principle is that it is a working set, not an archive: it holds only what must be queried quickly and immediately. Everything bulky or cold — original files, extracted text, archived vectors, archived transcripts, and backups — lives in Cloudflare R2. This inversion, treating R2 as the database and Neon as a fast cache, is what allows a feature-complete system to run for years within a 0.5 GB active limit.

5.2 Entity Relationship Design

The core entities and their relationships are as follows. A user owns everything: chats, messages, documents, collections, memories, search queries, agents, and knowledge-graph nodes. A chat has many messages. A document belongs to one or more collections and is decomposed into many document chunks, each carrying one vector. A collection has many chunks through its documents. Memories, provider-usage records, provider-health records, and audit logs are owned by the user but otherwise standalone. Agents own agent runs. Knowledge-graph entities are connected by knowledge-graph relations.

5.3 Table Structures

The complete table catalogue appears in Appendix C. The principal tables and their roles are summarised below.

Table	Role and key columns
users	Identity and profile; email, password hash, role, verification flag, timestamps.
refresh_tokens	Hashed opaque refresh tokens with expiry and revocation; purged on expiry.
chats	Conversation containers; title, archive flag, archived-transcript R2 key.
messages	Conversation turns; role, content, provider, model, token count, completeness flag; rolling window.
documents	Thin metadata only; filename, mime, sizes, content hash, R2 original key, R2 text key, status.
collections	Knowledge bases; name, last-accessed timestamp, archive flag, archived R2 key.
document_chunks	Half-precision vector, text preview and R2 pointer, embedding

Table	Role and key columns
	model and dimension.
memories	Curated long-term facts; type, content, importance, confidence, frequency, validity interval, embedding.
provider_usage	Per-call telemetry; rolled up nightly to daily aggregates and pruned.
provider_health	Provider state and cooldown for health-aware routing.
audit_logs	Security-relevant events; retained hot for a bounded window, then archived.
search_queries	Search history metadata; result bodies stored in R2.
agents / agent_runs	Agent definitions and bounded run records with trace pointers.
kg_entities / kg_relations	Personal knowledge graph (roadmap, Phase 9): entities and typed, temporal relations.

5.4 Relationships

Every owned table carries a user identifier as a foreign key, which is the single abstraction expressing ownership and the only generalisation needed toward a hypothetical multi-user future. Messages reference their chat; chunks reference their document and collection; agent runs reference their agent; knowledge-graph relations reference subject and object entities. Referential integrity is enforced with foreign-key constraints, and cascading deletes remove dependent rows when a parent is hard-deleted.

5.5 Indexing Strategy

Indexes are chosen for the two dominant access patterns: time-windowed reads and vector similarity search. The messages table is indexed on chat and creation time to support the rolling window; the usage table is indexed on creation time to support nightly rollup; documents are indexed on content hash for deduplication. Vector search uses an HNSW index over the half-precision embedding column with cosine distance. Index overhead is counted explicitly against the storage budget, since on a 0.5 GB database an index is a material consumer, not a free optimisation.

5.6 Data Retention Strategy

Retention is the heart of the design and is specified as concrete, automated policy rather than aspiration. Neon retains only the active working set; everything else moves to R2 on deterministic triggers.

Data	Retention policy
Original files & extracted text	Written to R2 at upload; never stored in Neon; retained permanently in R2.
Document chunks / vectors	Active in Neon while their collection is used; a collection idle beyond sixty days, or chunk count near the active cap, triggers export of vectors plus pointers to R2 and hard deletion from Neon. Reactivation re-inserts from the export with no re-embedding.
Chat messages	Hot window of ninety days (or last N per chat) in Neon; older transcripts serialised to R2 and hard-deleted from Neon, with the chat still listed and lazily reloadable; an optional short summary is retained in Neon.
Memories	Retained permanently in Neon but curated: deduplicated on insert, scored, decayed, and contradiction-resolved by closing a validity interval rather than deleting.
Knowledge graph	Retained in Neon as the reasoning layer; stale low-confidence relations may be closed or archived if the graph approaches its budget.
provider_usage (raw)	Rolled up nightly to daily aggregates; raw rows then deleted.
refresh_tokens	Hard-deleted on expiry; never archived.
audit_logs	Hot for roughly ninety days; older entries archived to R2.

Soft deletion is deliberately avoided as a general strategy because, on a 0.5 GB budget, tombstone rows and the dead tuples they create consume scarce space; the system uses a brief grace window followed by hard deletion, with vacuuming after large deletions so that space is genuinely reclaimed. Certain data is never archived because it is small, security-critical, or load-bearing as an index into R2: user and authentication records, curated memories, high-confidence knowledge-graph relations, document metadata pointers, and the active chat window.

5.7 Backup Strategy

A weekly logical backup of the database is produced by a GitHub Actions workflow and stored, compressed, in R2 with multiple versions retained. The restore procedure is tested at least once into a separate Neon project before it is relied upon, on the principle that an untested backup is only a hypothesis. The system supports full recovery, database restore, and knowledge-base restore from these artefacts together with the durable content already held in R2.

Chapter 6 — AI & Intelligence Layer

6.1 AI Goals

The intelligence layer aims to deliver grounded, trustworthy responses that draw on the user's own corpus and memory, to remain independent of any single model vendor, and to operate within free-tier quotas through disciplined request management. Trustworthiness is treated as an explicit engineering goal: the system prefers an honest abstention to a confident fabrication, and it makes the basis of an answer inspectable.

6.2 Model Architecture

Inference is served by three providers in priority order. Gemini Flash is the primary model for chat and the sole provider for embeddings, chosen for its generous free quota and free, high-throughput embedding access. Groq is the secondary provider, valued for low-latency open-model inference. OpenRouter is the tertiary, last-resort provider, offering access to free community models through a unified API. Provider and model names are held in configuration, never inline, so that the frequent deprecation of model versions is absorbed by a configuration change rather than a code change.

6.3 Routing Layer

The routing layer within the Gateway selects a provider using configured priority and live health. Each provider has a health state — healthy, rate-limited, temporarily failed, or disabled — and an associated cooldown. Health is updated reactively: a rate-limit or server error places a provider into cooldown, during which it is skipped, and it returns to service when the cooldown expires. Reactive health is preferred over active periodic probing because the application instance sleeps on the free tier, so a scheduled in-process probe would not run reliably; where continuous probing is ever required, it is performed by an external scheduled workflow rather than inside the application.

6.4 RAG Pipeline

Retrieval-augmented generation proceeds from upload through extraction, chunking, embedding, retrieval, context assembly, and grounded generation. Embeddings are 768-dimensional and stored in half precision to halve their footprint with negligible loss of recall. Three quality safeguards are built in from the outset: a relevance floor that produces an explicit abstention when no sufficiently similar chunk exists; an invariant that the query embedding must use the same model and dimension as the stored chunks, preventing the silent comparison of incompatible vector spaces; and a bounded context

budget that caps the number of chunks injected into any single generation. Citations resolve to a specific document and chunk, with full text loaded on demand from R2.

6.5 Memory System

Memory is divided into short-term and long-term forms. Short-term memory is simply the active conversation window and is not separately stored. Long-term memory holds durable preferences, facts, goals, and instructions in a curated table. The system's intelligence lies in keeping memory small and clean: new memories are deduplicated on insertion by semantic similarity, incrementing a frequency and confidence counter rather than creating duplicates; contradictions are resolved by closing the validity interval of the superseded memory rather than deleting it, which doubles as an audit trail; and retrieval applies importance scoring, recency decay, a similarity floor, and a hard cap on the number of memories injected per prompt. Without this curation, a growing memory store would degrade answer quality; with it, memory remains a net asset indefinitely.

6.6 Prompt Engineering Framework

Prompts are assembled by services, not by the Gateway, which preserves the boundary between provider communication and business logic. Assembled prompts clearly delimit instruction from retrieved content, so that text drawn from documents, knowledge bases, or search results is presented as data rather than as instructions. This delimitation is the first line of defence against prompt injection and becomes especially important once agents can take actions. Prompts are constructed to respect token budgets and to include only the highest-value memories and chunks.

6.7 Context Management

Context for any generation is assembled from three sources — the recent conversation window, retrieved knowledge-base chunks, and scored long-term memories — under an explicit token budget. The assembler favours recency and relevance, caps each source's contribution, and prefers to omit marginal context rather than overflow the budget. This bounded assembly is what allows a single Flash-class model with a generous context window to serve grounded answers without exhausting per-minute token limits.

6.8 Safety Layer

Safety spans both content and operation. Retrieved content is treated as untrusted and delimited as data. Agents operate under a strict tool allowlist, a hard cap on iterations, repetition detection, and a per-day request budget kept separate from interactive chat so that a runaway agent cannot exhaust the daily quota and lock the user out of the rest of the

system. Any destructive or irreversible tool requires explicit human confirmation. These measures convert the most dangerous failure modes — quota self-denial-of-service and injection-driven actions — from likely incidents into guarded edge cases.

6.9 Future AI Enhancements

Planned enhancements that fit the existing architecture without new infrastructure include a Personal Knowledge Graph with graph-augmented retrieval; a semantic response cache keyed by query-embedding similarity, which both demonstrates a cost-optimisation pattern and relieves the primary provider's daily quota; hybrid retrieval combining vector and full-text search to recover exact-match precision; an explainability layer that surfaces the chunks, memories, provider, and confidence behind each answer; and calibrated confidence scoring across memories, relations, and answers. Each reuses existing tables and remains free to operate.

Chapter 7 — Security Strategy

7.1 Authentication

Authentication is stateless and token-based. On registration or login the system issues a short-lived JWT access token, returned in the response body and held only in browser memory, and a long-lived opaque refresh token, stored hashed in the database and delivered as a Secure, HttpOnly cookie. The access token authorises every API call through a bearer header; the refresh token is used solely to obtain new access tokens. Passwords are hashed with bcrypt; refresh tokens are random and stored only as hashes, so that even a database disclosure exposes no usable token.

7.2 Authorization

Authorization is expressed through a single role field with two values, user and administrator, both held by the same person. The administrative surface introduced in Phase 6 is gated on the administrator role. Because the system is single-user, authorization is intentionally minimal, but the role field and the per-record user-ownership foreign key together constitute a clean authorization seam that could be elaborated without redesign.

7.3 Session Management

Sessions are carried entirely by the access token; only refresh-token validation touches the database. Refresh tokens rotate on every use: the presented token is revoked and a new one issued, so a stolen token is usable at most once before rotation invalidates it. Because the frontend and backend are on different origins, the refresh cookie is configured SameSite=None with Secure; the concurrent-refresh race that rotation can introduce is neutralised by single-flight refresh on the client and a short server-side grace window. When the in-memory access token is lost on reload, the application silently refreshes using the cookie to restore the session.

7.4 Encryption

All traffic is served over HTTPS by the hosting platforms, so tokens and data are encrypted in transit. Secret cookies carry the Secure attribute and are therefore never sent over plaintext connections. Passwords and refresh tokens are stored only in hashed form. Application secrets — database credentials, provider API keys, storage keys, and signing secrets — are held exclusively in environment variables and platform secret stores, never in source control.

7.5 API Security

Every request body, query parameter, and uploaded file is validated server-side against a strict schema before use; client input is never trusted. File uploads are validated for type and size and rejected on extension or content-type mismatch. Object storage is never public; files are served through short-lived signed URLs. Cross-origin requests are restricted to the specific frontend origin with credentials enabled, rather than a wildcard, which is required for cookie-bearing requests to be both functional and safe.

7.6 Rate Limiting

Per-user rate limiting is implemented with a database-backed counter rather than an external cache, in keeping with the prohibition on additional infrastructure. A separate, stricter daily budget governs agent activity, isolated from interactive chat so that automated loops cannot consume the entire daily provider quota. Provider-level cooldown, exponential backoff with jitter, and a per-run request cap further protect both the user and the upstream quotas from retry storms.

7.7 Audit Logging

Security-relevant events are recorded in an append-only audit log with a bounded hot-retention window, after which older entries are archived to R2. Application logs are structured and carry request identifiers for traceability, and a scrubbing step removes authorization headers and secrets before any log line is written, so that credentials are never persisted in logs.

7.8 Compliance Considerations

As a single-user, non-commercial system processing only its operator's own data, PrimeX AI is outside the scope of consumer data-protection regimes that govern multi-user services. One privacy consideration is nonetheless documented explicitly: free-tier inference may use submitted prompts to improve the provider's models. Because the system processes the operator's personal documents and memory, this trade-off is acknowledged as the non-monetary price of zero-cost operation, and the only mitigation that removes it — enabling paid billing — is incompatible with the zero-cost constraint and therefore deliberately not adopted.

Chapter 8 — Implementation Details

8.1 Frontend Modules

The frontend is a Next.js application written in TypeScript and styled with Tailwind CSS and shadcn/ui components. It is organised by feature: authentication, chat, documents, collections, memory, search, administration, agents, and settings. Client state is split by responsibility — an in-memory store holds ephemeral UI and authentication state, while a server-state library manages all data derived from the backend with caching, background refetch, and optimistic updates. An HTTP client centralises authorization and transparent token refresh; streaming responses are consumed incrementally and rendered as they arrive.

8.2 Backend Modules

The backend is a FastAPI application in Python. Its modules mirror the service architecture: core (configuration, database, security, logging, exceptions, middleware), models, schemas, repositories, services, the AI Gateway and its provider adapters, the RAG components, the storage adapter, background task functions, and the API routers. Each feature is implemented as a vertical slice through these layers, and each module is designed so that, if it ever became a bottleneck, it could be extracted to a standalone service through its existing service interface.

8.3 API Layer

The API is versioned under a single prefix and is resource-oriented, expressing actions through HTTP methods rather than verbs in paths. The complete endpoint catalogue appears in Appendix B. Authentication endpoints issue and refresh tokens; chat endpoints manage conversations and stream responses; document, collection, memory, search, administration, and agent endpoints expose the corresponding features. Every protected endpoint requires a bearer access token; the refresh endpoint alone consumes the refresh cookie.

8.4 Storage Layer

Object storage is Cloudflare R2, accessed through an S3-compatible adapter. R2 holds original files, extracted text, archived vectors, archived transcripts, search-result bodies, and database backups under a deterministic key layout that mirrors the logical structure of the data. Files are served to the browser through short-lived signed URLs so that file traffic never passes through, or counts against, the application instance's bandwidth. The

relational database holds only pointers and active working data, making R2 the system of record and Neon its queryable index.

8.5 Background Jobs

Scheduled and heavy background work is performed by GitHub Actions workflows, because the free application tier provides no separate worker and sleeps when idle. Workflows perform nightly usage rollup, daily vector archival, transcript archival, weekly backups, and weekly orphan reconciliation between R2 and Neon. Lightweight, request-triggered work uses the framework's in-process background tasks, made idempotent and re-triggerable through a status field so that a task lost to an instance restart can be safely replayed.

8.6 Event Processing

The system does not adopt a general event-sourcing architecture, which would be hostile to the storage budget. Instead it applies append-only, event-style modelling in a single bounded domain — the audit log, and optionally the memory subsystem — where the ability to replay and to query history over time provides real value at a controlled storage cost. This bounded approach demonstrates the pattern without paying its full storage price across the whole system.

8.7 Deployment Pipeline

Continuous integration runs linting and the test suite on every pull request. On merge to the main branch, the frontend deploys automatically to Vercel and the backend to Render. Database schema changes are applied through versioned migrations, validated first against a separate staging database before being applied to production. Environment variables and secrets are configured per platform and per GitHub Actions workflow, never committed to the repository.

Chapter 9 — Testing Strategy

9.1 Unit Testing

Unit tests cover the pure business logic of services, repositories, and utilities in isolation: token generation and verification, password hashing, chunking, memory deduplication and scoring, prompt assembly, and the Gateway's provider-selection and fallback logic with providers mocked. The layered architecture, which keeps business rules free of transport and persistence concerns, is what makes this logic cheaply unit-testable.

9.2 Integration Testing

Integration tests exercise complete vertical slices against a real staging database: the full authentication lifecycle, the document upload-and-extract pipeline, the end-to-end RAG path from ingestion to cited answer, memory creation and recall, and provider fallback against a simulated outage. These tests confirm that the layers cooperate correctly and that migrations apply cleanly.

9.3 End-to-End Testing

End-to-end tests validate the deployed system across the origin boundary: cross-domain login and refresh on the real frontend and backend URLs, streaming chat rendered incrementally in the browser, document upload and querying, and an agent run completing within its budget. The first end-to-end milestone — deployed cross-domain authentication — is treated as a gate that must pass before further feature work proceeds.

9.4 Security Testing

Security testing verifies that protected endpoints reject missing or invalid tokens, that refresh-token rotation invalidates a reused token, that the concurrent-refresh race does not cause spurious logout, that input and file validation reject malformed and oversized inputs, that signed URLs expire, and that retrieved content cannot induce an agent to call a tool. Secret scrubbing in logs is verified explicitly.

9.5 Performance Testing

Performance testing focuses on the characteristics that matter for a single user: time-to-first-token for streamed responses, retrieval latency over the vector index, and the memory footprint of document extraction. The objective is not high throughput but staying within the application instance's memory ceiling and providing responsive interaction once warm.

9.6 Load Testing

Formal load testing is scoped to the realistic single-user envelope rather than concurrent-user stress. The relevant load scenarios are batch operations that one user can trigger — bulk document ingestion and the resulting burst of embedding calls — which are tested against provider per-minute and per-day limits and against the application's memory ceiling, since these, not concurrency, are the true limits of the system.

9.7 User Acceptance Testing

Acceptance testing is performed by the operator against each phase's definition of done: that authentication survives a refresh, that chat streams and persists, that a document can be summarised and queried, that a collection yields cited answers and abstains when appropriate, that memories recall across sessions, that fallback is invisible, that the dashboard reflects reality, and that voice and agents function within their guardrails.

Chapter 10 — Deployment Architecture

10.1 Development Environment

Development is local: the Next.js frontend and FastAPI backend run on the developer's machine against a dedicated development database project and a separate development object-storage bucket, so that experimentation never touches production data. The toolchain comprises a current Node.js LTS, Python 3.12 or later, Git, and a code editor, all free.

10.2 Staging Environment

Staging is provided by a second free database project, used as the target for validating migrations and integration tests before production. Because the free database tier permits multiple projects, a full staging database is available at no additional cost, which removes the considerable risk of applying unvalidated migrations directly to production.

10.3 Production Environment

Production comprises the frontend on Vercel, the backend on Render, the database on Neon, object storage on Cloudflare R2, and monitoring on Sentry. Each is consumed within its free tier. File serving is routed through R2 signed URLs to keep bandwidth off the application instance, and scheduled work runs on GitHub Actions because the production application tier offers no worker process.

10.4 CI/CD Pipeline

The pipeline is built on GitHub. Pull requests trigger linting and tests; merges to the main branch trigger automatic deployment of both services. Scheduled workflows operate the storage lifecycle and backups. This arrangement provides a complete continuous-integration and continuous-deployment capability, plus an operational automation platform, entirely within the free GitHub Actions allowance.

10.5 Infrastructure Layout

Concern	Provider and configuration
Frontend hosting	Vercel (Hobby): Next.js application, served over HTTPS, non-commercial use.
Backend hosting	Render (Free web service): FastAPI, 512 MB memory, sleeps when idle, no persistent disk.
Relational + vector DB	Neon (Free): PostgreSQL with pgvector, 0.5 GB per project, scale-to-zero, separate staging project.

Concern	Provider and configuration
Object storage	Cloudflare R2 (Free): 10 GB, zero egress, deterministic key layout, signed-URL access.
Automation	GitHub Actions: CI, archival, rollup, backup, and reconciliation workflows.
Monitoring	Sentry (Free): exception and performance signals from client and backend.
Inference	Gemini (primary + embeddings), Groq (secondary), OpenRouter (tertiary), all free tiers.

10.6 Monitoring & Alerting

Operational visibility comes from three sources: per-request usage telemetry rolled up daily, the administrative dashboard that reads usage, storage, and health from existing tables, and Sentry for exceptions and performance. The single most important alert is the storage threshold: when the active database crosses eighty per cent of its budget, the system escalates archival so that it never approaches the hard limit unprepared.

10.7 Disaster Recovery

Disaster recovery rests on the durability of R2 and the weekly database backups stored there. Because original files and extracted text are the system of record and live permanently in R2, and because embeddings are explicitly regenerable, a catastrophic loss of the active database is a recoverable performance incident rather than a data-loss event: the database schema is restored from the latest backup and, where necessary, embeddings are regenerated from the preserved text through an external batch job that respects provider quotas. The restore path is tested before it is needed.

Chapter 11 — Project Roadmap

The roadmap delivers all eight approved phases in sequence. Each phase ships usable capability and prepares the foundation for the next; no phase is a throwaway. The first three phases are strictly sequential and load-bearing, since they establish authentication, the storage model, and retrieval. For each phase the goals, deliverables, dependencies, risks, and exit criteria are stated below.

Phase 1 — Foundation: Authentication, Chat, Gateway, and Storage Foundations

Goals: Establish the deployed platform: secure authentication, streaming chat served through the AI Gateway with provider fallback, usage tracking, monitoring, and the storage and automation foundations every later phase depends on.

Deliverables:

- Cross-origin authentication with rotating refresh tokens.
- Streaming multi-turn chat with persistent, paginated history.
- AI Gateway with Gemini primary and Groq fallback, and per-call usage logging.
- Sentry monitoring, structured logging, and the R2 storage adapter.
- GitHub Actions workflows for nightly usage rollup and weekly backup.

Dependencies: None.

Risks: Cross-domain authentication failing after build, and streaming timing out if routed through a serverless proxy.

Exit Criteria: The deployed system supports register, login, refresh, and streaming chat; a simulated provider outage is served by the fallback; rollup and backup workflows run green and a restore has been tested.

Phase 2 — Document Intelligence

Goals: Allow documents to be uploaded, stored durably in object storage, extracted, summarised, and queried, while establishing the index-and-store split that underpins the storage strategy.

Deliverables:

- Upload of PDF, DOCX, and TXT with original and extracted text stored in R2.
- Thin document metadata in the database with status tracking.

- Single-document summarisation and grounded question answering.
- Weekly orphan-reconciliation workflow between R2 and the database.

Dependencies: Phase 1 Gateway, storage adapter, and authentication.

Risks: Large or scanned files exhausting the application instance's memory during extraction.

Exit Criteria: A document can be uploaded, stored, extracted, summarised, and questioned with answers verifiably grounded in its content; reconciliation runs green.

Phase 3 — Knowledge Bases and RAG

Goals: Organise documents into collections, chunk and embed their text, retrieve by vector similarity, answer with citations, and ship the vector-archival engine that keeps the active database bounded.

Deliverables:

- Collections, chunking, half-precision embedding, and pgvector retrieval.
- Cited, grounded answers with an explicit abstention path for weak retrieval.
- Daily vector-archival workflow that evicts cold collections to R2 and reactivates them without re-embedding.

Dependencies: Phase 2 extraction pipeline; Phase 1 Gateway extended with embedding.

Risks: Silent retrieval-quality failure from a missing relevance floor or an embedding-space mismatch.

Exit Criteria: A collection yields grounded, cited answers, abstains on weak queries, and an idle collection archives to R2 and reactivates with no re-embedding.

Phase 4 — Memory System

Goals: Provide durable, curated long-term memory that is deduplicated, scored, contradiction-aware, and injected into prompts to personalise responses across sessions.

Deliverables:

- Memory creation, retrieval, update, and deletion with semantic deduplication.
- Contradiction resolution by validity interval, and scored, floored retrieval.
- Manual memory first, with automatic extraction built but feature-gated.

Dependencies: Phase 3 embedding capability and the Gateway.

Risks: Memory becoming noisy as it grows, degrading rather than improving answers.

Exit Criteria: A fact stated in one session is recalled unprompted in a later one; duplicates collapse and contradictions supersede cleanly.

Phase 5 — Provider Routing

Goals: Mature the linear fallback into health-aware routing with cooldown states and add OpenRouter as a tertiary provider.

Deliverables:

- Provider health table with healthy, rate-limited, failed, and disabled states.
- Reactive cooldown and OpenRouter integration as last resort.
- Routing that consults health before dispatch.

Dependencies: Phase 1 Gateway interface, unchanged except in selection logic.

Risks: Retry storms hammering an already-failing provider.

Exit Criteria: A forced rate-limit cools the primary, routes to the secondary, and recovers after the cooldown; the tertiary serves when both are down.

Phase 6 — Search and Administration

Goals: Add web search with resolvable citations and an operational dashboard built from existing telemetry.

Deliverables:

- Search via a free search API with citation resolution and persisted history.
- Administrative dashboard for usage, storage, and provider health.

Dependencies: The Gateway, the usage tables, and the RAG citation pattern.

Risks: Accidentally adopting a billed search path, and citations resolving to dead links.

Exit Criteria: A search returns an answer with at least one resolvable citation; the dashboard reflects real usage, storage, and health from existing tables.

Phase 7 — Advanced File Intelligence

Goals: Extend the document pipeline to ZIP, CSV, XLSX, and PPTX formats with no new architecture.

Deliverables:

- Format handlers integrated into the existing extraction and chunking pipeline.
- Decompression and size guards to protect the memory-limited instance.

Dependencies: The Phase 2 extraction pipeline and Phase 3 chunking.

Risks: A compression bomb or oversized spreadsheet exhausting instance memory.

Exit Criteria: Each new format uploads, extracts, chunks, and is queryable through existing retrieval; malicious or oversized files are rejected gracefully.

Phase 8 — Voice and Agents

Goals: Add browser-based voice interaction and a bounded, tool-using agent runtime with strict safety and quota guardrails.

Deliverables:

- Browser speech recognition and synthesis wired to the existing chat endpoint.
- A constrained agent loop calling the system's own tools, with run tracing.
- Hard iteration caps, repetition detection, a separate daily agent budget, and human confirmation for destructive tools.

Dependencies: All prior phases; agents call the system's own RAG, search, and summarisation tools.

Risks: Quota self-denial-of-service from a runaway loop, and injection-driven tool calls.

Exit Criteria: Voice input and output function in the browser; an agent completes a multi-step task within its budget; an injected instruction in retrieved content does not trigger a tool call.

Phase 9 (Roadmap) — Personal Knowledge Graph

Beyond the eight committed phases, a Personal Knowledge Graph is documented as a roadmap item rather than a delivered phase. It would extract entities and relations from the accumulated corpus into graph tables within the existing database and support graph-augmented retrieval. It is deferred deliberately because it depends on a populated corpus to extract from, is the most complex single component, and requires a heavy one-time backfill that must run externally to respect provider quotas. Its placement as an explicit, scoped Phase 9 — rather than as vague future work — is itself a demonstration of deliberate scoping.

Chapter 12 — Engineering Decisions

This chapter records the major architectural decisions that define PrimeX AI. Each decision states the problem, the options considered, the final choice, its justification, and its trade-offs. Together they constitute the rationale that protects the architecture against future second-guessing and rewrites.

Decision 12.1 — Storage Inversion: Neon as Index, R2 as System of Record

Problem: A feature-complete AI platform generates far more data than a 0.5 GB database can hold, yet must remain queryable and free.

Options Considered: Store everything in the database and upgrade when full; store everything in object storage and query slowly; or split the two by access pattern.

Final Choice: Treat the database as a bounded active working set and object storage as the durable system of record, with the database holding only pointers and hot data.

Justification: Object storage at 10 GB with zero egress is effectively unlimited for one user, so unbounded data lives there while the queryable set in the database stays within budget; originals and text become the source of truth and embeddings become regenerable.

Trade-Offs: Adds an archival subsystem and cross-store consistency concerns; some reads require fetching text from object storage; the model must be built correctly from the first phase.

Decision 12.2 — Cross-Origin Authentication Topology

Problem: The frontend and backend are served from different origins, which breaks a conventional same-site refresh cookie.

Options Considered: Same-site strict cookie (broken across origins); store tokens in local storage (vulnerable to script access); bearer access token everywhere with a cross-site refresh cookie; or proxy all traffic through one origin.

Final Choice: Authenticate every call with an in-memory bearer access token, and use a SameSite=None, Secure refresh cookie solely on the refresh endpoint, with rotation protected by single-flight handling.

Justification: Bearer headers cross origins safely and keep the access token out of persistent storage; confining the cookie to one endpoint minimises its exposure; this preserves direct streaming from the backend without a proxy.

Trade-Offs: The refresh cookie is a third-party cookie that strict browsers may block, which is acceptable for a controlled single-user browser; rotation requires careful race handling.

Decision 12.3 — Half-Precision Embeddings at 768 Dimensions

Problem: Full-precision, high-dimensional vectors consume scarce database space and limit the active corpus.

Options Considered: Full-precision 3072-dimensional vectors; full-precision 768-dimensional vectors; or half-precision 768-dimensional vectors.

Final Choice: Standardise on 768-dimensional half-precision vectors, truncating the embedding model's output explicitly.

Justification: Half precision roughly halves vector storage for negligible recall loss, and a fixed 768-dimension contract keeps all chunks comparable in one index; the saving directly increases the active-corpus capacity within the budget.

Trade-Offs: Some recall is traded for capacity; changing the dimension later requires a full re-embedding migration, which the model-and-dimension columns are designed to make safe.

Decision 12.4 — GitHub Actions for Scheduled and Heavy Background Work

Problem: The free application tier provides no separate worker and sleeps when idle, so scheduled and heavy work cannot run reliably in-process.

Options Considered: Run scheduled work in-process (lost on sleep); add a paid worker (violates budget); introduce a queue such as Celery with a broker (prohibited infrastructure); or use the CI platform's scheduler.

Final Choice: Perform all scheduled and heavy work — archival, rollup, backup, reconciliation, and bulk re-embedding — as GitHub Actions workflows, reserving in-process tasks for lightweight, request-triggered, idempotent work.

Justification: The CI platform's scheduler is free, reliable, and external to the sleeping instance, and re-uses infrastructure already present for continuous integration.

Trade-Offs: Scheduled work runs on a fixed cadence rather than continuously, and heavy jobs are constrained by the workflow's own limits; truly continuous monitoring is not possible in-process.

Decision 12.5 — Vendor-Independent AI Gateway with Fallback Chain

Problem: Dependence on a single model provider is fragile, given documented free-tier quota cuts, outages, and frequent model deprecations.

Options Considered: Call a single provider directly; hardcode a primary with manual switching; or abstract all providers behind one gateway with automatic fallback and health.

Final Choice: Route all inference through a single AI Gateway that abstracts Gemini, Groq, and OpenRouter, selects by configured priority and live health, and falls through on classified failures.

Justification: The gateway makes the system vendor-independent, turns provider changes into configuration changes, and converts a provider outage from an application failure into a transparent fallback.

Trade-Offs: Adds an abstraction layer and the subtlety of mid-stream failover, which is resolved by failing over only before the first token and otherwise preserving partial output.

Decision 12.6 — Self-Hosted Stateless Authentication

Problem: Authentication must be secure and free, without a paid identity provider.

Options Considered: Adopt a managed authentication service (introduces a dependency and potential cost); or implement stateless JWT authentication with rotating refresh tokens.

Final Choice: Implement authentication in-house with short-lived JWT access tokens and rotating, hashed refresh tokens.

Justification: In-house authentication is free, fully under the project's control, and a strong demonstration of security engineering; statelessness keeps access-token validation off the database.

Trade-Offs: The project bears responsibility for correct implementation of hashing, rotation, and race handling, which is mitigated by following well-established patterns and testing them explicitly.

Decision 12.7 — API-Based Embeddings over Local Models

Problem: Embeddings must be produced cheaply without exceeding the application instance's memory ceiling.

Options Considered: Run a local embedding model in the application (memory-heavy on a 512 MB instance); or call the provider's free embedding API.

Final Choice: Generate all embeddings through the provider's free embedding API via the Gateway.

Justification: The free embedding API is generous and network-bound, avoiding the memory pressure that a local transformer model would place on the constrained instance, and keeps embedding consistent with the rest of the Gateway.

Trade-Offs: Embeddings depend on provider availability and quota, and a provider embedding-model deprecation would require a re-embedding migration, which is planned for as an external batch job.

Decision 12.8 — Hard-Delete Archival over Soft Deletion

Problem: A 0.5 GB database cannot afford to retain logically deleted rows and the dead tuples they create.

Options Considered: Soft-delete everything with tombstones; or use a brief grace window followed by hard deletion and archival.

Final Choice: Avoid general soft deletion; hard-delete and archive to object storage, vacuuming after large deletions, and express supersession through validity intervals where history matters.

Justification: Hard deletion reclaims scarce space, while validity intervals preserve the audit history that soft deletion is usually invoked to protect, at far lower cost.

Trade-Offs: Recovery of a deleted row relies on archives rather than an in-place flag, which is acceptable given that durable content already lives in object storage.

Decision 12.9 — pgvector over a Dedicated Vector Database

Problem: Vector search is required, but an additional managed vector service would add cost and another dependency.

Options Considered: Adopt a dedicated vector database; or use the pgvector extension inside the existing PostgreSQL.

Final Choice: Use pgvector within Neon PostgreSQL, with an HNSW index over half-precision vectors.

Justification: Co-locating vectors with relational data removes a moving part, keeps everything within one free database, and simplifies consistency between chunks and their metadata.

Trade-Offs: Vector storage and its index compete with relational data for the 0.5 GB budget, which the archival engine and half precision are designed to manage.

Decision 12.10 — Knowledge Graph Deferred to a Roadmap Phase

Problem: A personal knowledge graph is the project's strongest differentiator but also its most complex and corpus-dependent component.

Options Considered: Build the graph early as a committed phase; or scope it as a deliberate post-Phase-8 roadmap item.

Final Choice: Defer the knowledge graph to a scoped Phase 9, after a populated corpus exists, with its heavy backfill planned as an external job.

Justification: Building over an empty corpus would extract little; deferring lets the graph form over real accumulated data and keeps the committed timeline achievable, while explicit scoping signals deliberate prioritisation.

Trade-Offs: The highest-uniqueness capability arrives last and is not guaranteed within the core timeline, which is an accepted trade for protecting completion of the eight committed phases.

Chapter 13 — Risks & Mitigation

This chapter is the project risk register. Each risk is described with its impact, probability, the mitigation built into the architecture, and a contingency should the mitigation prove insufficient. The register is ordered by severity.

13.1 — Active-Database Storage Exhaustion

Description: Chat history, chunks, vectors, usage telemetry, and logs all share a single 0.5 GB database and, without intervention, fill it within months.

Impact: Critical: when full, writes fail and the system is effectively down.

Probability: Near-certain over time without mitigation.

Mitigation Plan: Bound every growing table: a rolling chat window, daily usage rollups, vector archival to object storage, hard deletion over soft deletion, half-precision vectors, and an eighty-per-cent storage alert that escalates archival early.

Contingency Plan: If the working set still approaches the limit, tighten retention windows and the active-vector cap, and archive more aggressively; the durable copy in object storage means no data is lost.

13.2 — Primary Provider Free-Tier Volatility

Description: The primary model provider has cut free quotas substantially and without notice in the recent past and may do so again.

Impact: High: degraded or unavailable inference across the whole system.

Probability: Medium and recurring.

Mitigation Plan: Treat multi-provider fallback as a Phase 1 requirement; route through the Gateway so a degraded primary falls through to secondary and tertiary providers automatically; track usage to detect quota pressure early.

Contingency Plan: If all free inference degrades simultaneously, reduce request volume through semantic caching and tighter context budgets; the architecture's provider independence allows new free providers to be added through configuration.

13.3 — Loss of In-Process Background Tasks

Description: The application instance sleeps and restarts, so in-process background tasks can be silently dropped before completion.

Impact: High: inconsistent state from unfinished file processing or memory extraction.

Probability: High under normal free-tier operation.

Mitigation Plan: Move scheduled and heavy work to external workflows; make in-process tasks idempotent and re-triggerable through a status field; add a reconciliation sweep for cross-store orphans.

Contingency Plan: Re-trigger any task left in a processing state; reconciliation detects and repairs orphans between the database and object storage.

13.4 — Silent Retrieval-Quality Failure

Description: Vector retrieval can return irrelevant context with no error, and an embedding-space mismatch can produce meaningless similarity, leading to confident but wrong answers.

Impact: High: erodes the core value of grounded answering without any visible failure.

Probability: High if unguarded.

Mitigation Plan: Enforce a relevance floor with explicit abstention, require the query embedding to match the chunks' model and dimension, instrument retrieval scores, and add a groundedness self-check.

Contingency Plan: If quality regresses, introduce hybrid vector-and-text retrieval and a real reranker, both of which the architecture already accommodates.

13.5 — Memory Degradation

Description: An accumulating memory store fills with duplicates and contradictions and injects noisy context, making answers worse over time.

Impact: High: the intended differentiator becomes a liability.

Probability: High over time without curation.

Mitigation Plan: Deduplicate on insert, resolve contradictions by validity interval, score by importance with recency decay, and enforce a similarity floor and a per-prompt cap on injected memories.

Contingency Plan: If noise persists, raise the dedup and retrieval thresholds and reduce the per-prompt memory budget.

13.6 — Agent Quota Self-Denial-of-Service

Description: An uncapped agent loop can exhaust the entire daily inference quota in minutes, locking the user out of all features until the quota resets.

Impact: High: total loss of interactive function for the remainder of the day.

Probability: Medium.

Mitigation Plan: Cap agent iterations, detect repetition, and enforce a daily agent budget kept separate from interactive chat, with a circuit breaker.

Contingency Plan: If the budget is exhausted, agents are paused for the day while interactive chat continues on its own reserved quota.

13.7 — Cross-Store Consistency Orphans

Description: Writing to object storage and the database is not a single transaction, so a partial failure can leave an orphaned file or orphaned metadata.

Impact: Medium to high: wasted storage or broken downloads.

Probability: Medium.

Mitigation Plan: Record intent before acting, treat object storage as eventually consistent, and run a weekly reconciliation that detects and resolves orphans in both directions.

Contingency Plan: Reconciliation cleans orphaned objects and repairs or removes orphaned metadata on its scheduled run.

13.8 — Prompt Injection via Retrieved Content

Description: Malicious instructions embedded in a document, knowledge-base chunk, or search result could hijack a response or, with agents, an action.

Impact: Medium for answers; high once agents can act.

Probability: Medium.

Mitigation Plan: Delimit all retrieved content as data, restrict each agent to an allowlist of tools, and require human confirmation for any destructive tool.

Contingency Plan: If an injection is detected, the offending source is quarantined and the agent's tool set is further restricted.

13.9 — Provider Model Deprecation and Dependency Drift

Description: Model versions are deprecated frequently, and unattended dependencies accumulate breaking changes over a year.

Impact: Medium: broken inference or build, requiring maintenance.

Probability: Medium to high over a year.

Mitigation Plan: Hold model names in configuration; keep the provider abstraction; schedule periodic dependency review so upgrades are incremental rather than a single large jump.

Contingency Plan: Repin to a current model in configuration; address dependency breakage in a focused maintenance window.

13.10 — Cold-Start Latency Perceived as Failure

Description: Stacked sleep of the application instance and the database produces a multi-second first response that can appear broken.

Impact: Low to medium: a user-experience tax, not a fault.

Probability: High but benign.

Mitigation Plan: Accept cold starts as a free-tier characteristic, communicate loading state clearly in the interface, and never self-ping to stay warm, since that risks suspension and burns the instance-hour budget.

Contingency Plan: If latency becomes intolerable, the only remedy within the constraints is to reduce idle frequency through normal use; paid warm instances are out of scope by assumption.

Chapter 14 — Project Management

14.1 Team Structure

The project is executed by a single developer who holds every engineering role in sequence: architect, backend engineer, frontend engineer, AI engineer, database engineer, security engineer, DevOps engineer, and reviewer. This consolidation is appropriate for a personal learning project and is itself part of its value, since completing the full breadth of the system alone is the demonstration the project exists to provide.

14.2 Roles & Responsibilities

Role	Responsibilities
Architect	Owns the locked architecture, the storage lifecycle, and decision consistency.
Backend engineer	Implements services, repositories, the API, and the RAG and memory engines.
Frontend engineer	Implements the Next.js interface, state management, and streaming UI.
AI engineer	Implements the Gateway, provider adapters, routing, prompts, and agents.
Database engineer	Owns the schema, migrations, indexing, and retention policy.
Security engineer	Owns authentication, validation, rate limiting, and audit logging.
DevOps engineer	Owns deployment, CI/CD, and the GitHub Actions automation.
Reviewer / QA	Owns testing, definitions of done, and acceptance against each phase.

14.3 Development Workflow

Work proceeds in vertical slices, one feature at a time through all layers, on short-lived branches merged through pull requests that trigger linting and tests. The schema is created generously up front so that later phases add behaviour without migrating structure, and code is kept deliberately lean to preserve velocity. Each phase is kept individually shippable so that progress is continuously demonstrable and momentum — the principal risk in a long solo build — is protected.

14.4 Release Workflow

Releases are continuous: a merge to the main branch deploys both services automatically. Database migrations are validated against the staging project before production, and scheduled workflows operate the storage lifecycle independently of releases. A phase is considered released when it meets its definition of done on the deployed system.

14.5 Documentation Strategy

Documentation is treated as a first-class deliverable. The architecture, database, gateway, RAG, memory, storage, deployment, security, roadmap, decisions, and risks are recorded as source-of-truth documents, consolidated into this specification. The API documents itself through its schema, and a decision log preserves the rationale behind each major choice so that future maintenance does not relitigate settled questions.

14.6 Quality Assurance Process

Quality is assured by a layered test suite, a deployment gate on cross-domain authentication, explicit definitions of done per phase, and acceptance testing by the operator. The storage lifecycle is verified operationally — archival, rollup, backup, and a tested restore — because in this project correct data-lifecycle behaviour is as important as correct feature behaviour.

Chapter 15 — Future Enhancements

Short-Term Enhancements

Immediate enhancements reuse existing tables and remain free while repairing identified weaknesses. A semantic response cache keyed by query-embedding similarity demonstrates a cost-optimisation pattern and relieves the daily inference quota. Hybrid retrieval combining vector and full-text search restores exact-match precision. An explainability layer surfaces the chunks, memories, provider, and confidence behind each answer. Calibrated confidence scoring attaches uncertainty to memories, relations, and answers.

Mid-Term Enhancements

Mid-term work centres on the Personal Knowledge Graph: entity and relation extraction over the accumulated corpus, entity resolution, and graph-augmented retrieval that combines traversal with vector search. Temporal and bitemporal modelling of memories and relations enables historical queries about what the user knew or believed at a given time, demonstrating advanced data modelling at low storage cost.

Long-Term Vision

The long-term vision is a genuinely personal intelligence: a system that not only answers from the user's corpus but reasons across it, tracks how the user's knowledge and goals evolve, reflects on the groundedness of its own answers, and surfaces connections the user has not made explicitly. Every element of this vision is reachable from the existing architecture without new infrastructure and without departing from the zero-cost, single-user constraints, which is the property the project was designed to preserve.

Chapter 16 — Conclusion

Project Summary

PrimeX AI is a complete, modular, vendor-independent personal AI Operating System, specified end-to-end across eight delivered phases and a scoped ninth roadmap phase, engineered to run for years at zero recurring cost for a single user. Its architecture inverts the usual relationship between database and object storage, abstracts all inference behind a single resilient gateway, grounds answers in the user's own corpus with explicit honesty about uncertainty, and curates a persistent memory that improves rather than degrades over time.

Achievements

The project demonstrates, in one coherent artefact, the full breadth of modern AI systems engineering: secure stateless authentication across origins, a resilient multi-provider gateway, retrieval-augmented generation with citations and abstention, a curated long-term memory, advanced document intelligence, safe autonomous agents, and — most distinctively — a data-lifecycle architecture that makes a sophisticated system genuinely free to operate within hard limits. The constraint engineering is not a limitation worked around but the central engineering accomplishment.

Expected Impact

For its single user, the system delivers a durable, owned, and continually useful personal knowledge platform. As a portfolio artefact, it provides credible evidence of senior-level judgement: the ability to design for constraints, to choose deliberately among real trade-offs, to protect a long-term architecture against rewrites, and to build the unglamorous reliability and data-lifecycle machinery that distinguishes production systems from demonstrations.

Final Remarks

PrimeX AI was conceived to be built once and built correctly. Every decision in this specification serves that aim: to deliver the complete vision, to keep it free and maintainable for one user indefinitely, and to do so without the rewrites that befall systems whose foundations were laid expediently. The architecture is frozen, the lifecycle is defined, and the path from the first deployed login to the final agent run is mapped. What remains is disciplined execution.

Appendix A — Complete Folder Structure

The repository is a monorepo containing the frontend and backend applications and the automation workflows. The structure below reflects the system at completion.

Backend (FastAPI):

- app/main.py — application entry point
- app/core/ — configuration, database, security, logging, exceptions, middleware
- app/models/ — user, refresh_token, chat, message, document, collection, document_chunk, memory, provider_usage, provider_health, audit_log, search_query, agent, agent_run
- app/schemas/ — request and response models per feature
- app/repositories/ — base plus one repository per aggregate
- app/services/ — auth, chat, document, collection, rag, memory, search, admin, agent
- app/gateway/ — base interface, ai_gateway, router, providers (gemini, groq, openrouter)
- app/rag/ — extractor, chunker, embedder, retriever, reranker
- app/storage/ — r2_storage adapter
- app/jobs/ — task functions (process_file, embed_document, extract_memory)
- app/api/v1/ — auth, chat, documents, collections, memory, search, admin, agents, health
- app/utils/ — tokens, scrub, helpers
- alembic/ — versioned migrations
- tests/ — unit and integration suites

Frontend (Next.js):

- app/ — routes for auth, chat, documents, collections, memory, search, admin, settings
- components/ — feature components (chat, documents, memory, search, voice, ui)
- services/api/ — HTTP client, streaming consumer, per-feature API modules
- stores/ — in-memory client state

- hooks/ — feature hooks
- lib/ — query client and utilities
- middleware.ts — route guard

Automation (.github/workflows):

- ci.yml — lint and tests on pull request
- usage-rollup.yml — nightly aggregation of usage telemetry
- vector-archival.yml — daily eviction of cold vectors to object storage
- backup.yml — weekly database backup to object storage
- orphan-reconciliation.yml — weekly cross-store consistency sweep

Appendix B — API Catalog

Method	Route	Auth	Purpose
POST	/api/v1/auth/register	Public	Create account; issue tokens
POST	/api/v1/auth/login	Public	Authenticate; issue tokens
POST	/api/v1/auth/refresh	Cookie	Rotate refresh; issue access token
POST	/api/v1/auth/logout	Bearer	Revoke refresh; clear cookie
GET	/api/v1/auth/me	Bearer	Current user profile
GET	/api/v1/chats	Bearer	List chats (paginated)
POST	/api/v1/chats	Bearer	Create chat
GET	/api/v1/chats/{id}	Bearer	Chat with recent messages
PATCH	/api/v1/chats/{id}	Bearer	Rename chat
DELETE	/api/v1/chats/{id}	Bearer	Delete chat and messages
POST	/api/v1/chats/{id}/messages	Bearer	Send message; stream reply
POST	/api/v1/documents/upload	Bearer	Upload; store to object storage
GET	/api/v1/documents	Bearer	List documents
GET	/api/v1/documents/{id}	Bearer	Metadata and signed URL
DELETE	/api/v1/documents/{id}	Bearer	Delete metadata; schedule cleanup
POST	/api/v1/documents/{id}/summary	Bearer	Summarise document
POST	/api/v1/documents/{id}/ask	Bearer	Single-document question answering
POST	/api/v1/collections	Bearer	Create collection
GET	/api/v1/collections	Bearer	List collections
POST	/api/v1/collections/{id}/documents	Bearer	Add document; chunk

Method	Route	Auth	Purpose
			and embed
POST	/api/v1/collections/{id}/query	Bearer	RAG query; cited answer
POST	/api/v1/collections/{id}/search	Bearer	Semantic chunk search
GET	/api/v1/memories	Bearer	List memories
POST	/api/v1/memories	Bearer	Create memory (deduplicated)
PATCH	/api/v1/memories/{id}	Bearer	Update memory
DELETE	/api/v1/memories/{id}	Bearer	Delete memory
GET	/api/v1/health/providers	Bearer	Provider health and cooldown
POST	/api/v1/search	Bearer	Web search with citations
GET	/api/v1/search/history	Bearer	Search history
GET	/api/v1/admin/usage	Admin	Usage analytics
GET	/api/v1/admin/storage	Admin	Storage utilisation
POST	/api/v1/agents	Bearer	Create agent
POST	/api/v1/agents/{id}/run	Bearer	Execute bounded agent run
GET	/api/v1/agents/{id}/runs	Bearer	Agent run history
GET	/api/v1/health	Public	System health

Appendix C — Database Catalog

Table	Growth profile	Retention
users	Static	Permanent in database
refresh_tokens	Bounded by rotation	Hard-deleted on expiry
chats	Slow	Permanent metadata; transcript may archive
messages	Fast, unbounded	Ninety-day hot window; older archived to object storage
documents	Slow (pointers)	Permanent metadata; never archived
collections	Slow	Permanent; archive flag for cold sets
document_chunks	Fast with corpus	Active until idle; archived to object storage
memories	Slow if deduplicated	Permanent, curated; never bulk-archived
provider_usage	Fastest (per call)	Rolled up nightly; raw rows deleted
provider_health	Static	Permanent; small
audit_logs	Steady	Ninety-day hot; older archived
search_queries	Slow	Metadata in database; bodies in object storage
agents / agent_runs	Slow	Definitions permanent; run traces to object storage
kg_entities / kg_relations	Roadmap	Permanent reasoning layer; stale relations closed

C.1 Detailed Column Schemas

The following column-level schemas define the principal tables. Every owned table carries a user foreign key and standard creation and update timestamps, which are omitted from the per-table listings below for brevity and stated once here as a convention. Identifiers are universally unique values generated by the database.

C.1.1 users

Column	Type	Constraints and notes
id	UUID	Primary key; database-generated.
email	VARCHAR(255)	Unique; not null; validated format.
password_hash	VARCHAR(255)	Not null; bcrypt; never plaintext.

Column	Type	Constraints and notes
display_name	VARCHAR(120)	Optional profile name.
role	VARCHAR(20)	Not null; default 'user'; values 'user' or 'admin'.
is_verified	BOOLEAN	Not null; default true for single-user operation.
created_at / updated_at	TIMESTAMPTZ	Not null; audit timestamps.

C.1.2 refresh_tokens

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key to users; indexed.
token_hash	VARCHAR(255)	Not null; hash of the opaque token; unique.
expires_at	TIMESTAMPTZ	Not null; rotation and expiry boundary.
revoked	BOOLEAN	Not null; default false; set true on rotation or logout.
created_at	TIMESTAMPTZ	Not null.

C.1.3 chats

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key to users; indexed.
title	VARCHAR(200)	Not null; default derived from first message.
message_count	INTEGER	Denormalised count maintained on write.
is_archived	BOOLEAN	Not null; default false; true when transcript moved to object storage.
archived_r2_key	VARCHAR(300)	Nullable; key of the archived transcript.
created_at / updated_at	TIMESTAMPTZ	Not null.

C.1.4 messages

Column	Type	Constraints and notes
id	UUID	Primary key.
chat_id	UUID	Foreign key to chats; indexed with created_at.
role	VARCHAR(16)	Not null; 'user' or 'assistant'.
content	TEXT	Not null; message body within the hot window.
provider	VARCHAR(32)	Provider that produced an assistant message.
model	VARCHAR(64)	Model that produced an assistant message.
token_count	INTEGER	Tokens consumed; feeds usage accounting.
is_complete	BOOLEAN	Not null; default true; false for interrupted streams.
created_at	TIMESTAMPTZ	Not null; rolling-window key.

C.1.5 documents

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key; indexed.
filename	VARCHAR(255)	Not null; original file name.
mime	VARCHAR(100)	Not null; validated content type.
size_bytes	BIGINT	Not null; enforced against the upload cap.
content_hash	VARCHAR(64)	Indexed; deduplication key.
r2_original_key	VARCHAR(300)	Not null; pointer to the original in object storage.
r2_text_key	VARCHAR(300)	Nullable until extraction; pointer to extracted text.
status	VARCHAR(20)	Not null; 'pending','processing','ready','failed'.
error	TEXT	Nullable; failure detail for the user.

C.1.6 collections

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key; indexed.
name	VARCHAR(200)	Not null.
last_accessed_at	TIMESTAMPTZ	Not null; archival idle trigger.
is_archived	BOOLEAN	Not null; default false.
archived_r2_key	VARCHAR(300)	Nullable; archived vector export key.

C.1.7 document_chunks

Column	Type	Constraints and notes
id	UUID	Primary key.
document_id	UUID	Foreign key to documents.
collection_id	UUID	Foreign key to collections; indexed.
chunk_index	INTEGER	Not null; ordinal within the document.
preview	VARCHAR(220)	Short display preview; full text in object storage.
r2_text_key	VARCHAR(300)	Pointer to the chunk's full text.
text_offset / text_length	INTEGER	Offset and length within the text object.
embedding	halfvec(768)	Half-precision vector; HNSW cosine index.
embedding_model	VARCHAR(100)	Not null; model identity for re-embedding safety.
embedding_dimension	INTEGER	Not null; fixed at 768 by check constraint.

C.1.8 memories

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key; indexed.
type	VARCHAR(24)	Not null; 'preference','fact','goal','instruction'.
content	TEXT	Not null; the memory statement.

Column	Type	Constraints and notes
importance	REAL	Score driving retrieval priority.
confidence	REAL	Confidence in the memory's accuracy.
frequency	INTEGER	Reinforcement count from deduplication.
embedding	halfvec(768)	For semantic retrieval and dedup.
valid_from / valid_to	TIMESTAMPTZ	Validity interval; supersession instead of deletion.
last_used_at	TIMESTAMPTZ	Drives recency decay.

C.1.9 provider_usage

Column	Type	Constraints and notes
id	UUID	Primary key.
user_id	UUID	Foreign key.
provider	VARCHAR(32)	Not null.
model	VARCHAR(64)	Not null.
operation	VARCHAR(16)	'chat' or 'embed'.
latency_ms	INTEGER	Observed latency.
tokens	INTEGER	Tokens consumed.
status	VARCHAR(16)	'success','fallback','fail'.
error_type	VARCHAR(64)	Nullable; classified error.
created_at	TIMESTAMPTZ	Not null; nightly-rollup key, then pruned.

C.1.10 provider_health / agents / agent_runs

Column	Type	Constraints and notes
provider_health.state	VARCHAR(20)	'healthy','rate_limited','temporary_failure','disabled'.
provider_health.cooldown_until	TIMESTAMPTZ	Skip-until boundary for routing.
agents.allowed_tools	JSONB	Tool allowlist for the agent.
agents.system_prompt	TEXT	Agent instruction template.
agent_runs.status	VARCHAR(20)	'running','completed','failed','aborted'.
agent_runs.iterations	INTEGER	Steps taken; capped.

Column	Type	Constraints and notes
agent_runs.token_budget_used	INTEGER	Against the daily agent budget.
agent_runs.r2_trace_key	VARCHAR(300)	Pointer to the archived run trace.

Appendix D — Security Checklist

- Passwords hashed with bcrypt; never stored in plaintext.
- Refresh tokens random, stored hashed, and rotated on use.
- Access token short-lived and held only in browser memory.
- Refresh cookie HttpOnly, Secure, SameSite=None, scoped to the refresh endpoint.
- Single-flight refresh and a server grace window to neutralise the rotation race.
- All traffic over HTTPS; secret cookies never sent over plaintext.
- All input and uploaded files validated server-side; client input never trusted.
- Object storage private; files served via short-lived signed URLs.
- Cross-origin requests restricted to the specific frontend origin with credentials.
- Per-user rate limiting and a separate, stricter daily agent budget.
- Retrieved content delimited as data; agents restricted to a tool allowlist with human confirmation for destructive tools.
- Secrets only in environment variables and platform secret stores; scrubbed from logs.
- Append-only audit logging with bounded hot retention.

Appendix E — Deployment Checklist

- Neon production and staging projects provisioned; pgvector enabled.
- Object-storage production and development buckets created with deterministic key layout.
- Backend deployed to Render; frontend deployed to Vercel; environment variables set on each.
- Cross-origin authentication verified on the deployed URLs as the release gate.
- Streaming verified directly from the backend, not through a serverless proxy.
- GitHub Actions secrets configured; CI, rollup, archival, backup, and reconciliation workflows enabled.
- Sentry connected for both client and backend.
- Eighty-per-cent storage alert configured.
- Weekly backup produced and a restore tested into the staging project.
- File serving confirmed to route through signed URLs, keeping bandwidth off the instance.

Appendix F — Testing Checklist

- Authentication lifecycle: register, login, refresh, rotate, logout.
- Concurrent-refresh race handled without spurious logout.
- Gateway fallback serves a request when the primary provider fails.
- Streaming happy path and mid-stream failure with preserved partial content.
- Document pipeline: upload, extract, summarise, single-document question answering.
- RAG: grounded cited answer, abstention on weak retrieval, embedding-space match enforced.
- Memory: cross-session recall, deduplication, contradiction supersession.
- Advanced formats: ZIP, CSV, XLSX, PPTX processed; malicious or oversized files rejected.
- Agents: bounded run completes; injected instruction does not trigger a tool call; budget enforced.
- Storage lifecycle: archival, rollup, reconciliation, and restore verified operationally.

Appendix G — Glossary

AI Gateway: The single backend component through which all model calls pass, abstracting providers behind a uniform interface with fallback and health.

RAG: Retrieval-Augmented Generation: grounding model answers in retrieved passages from the user's corpus.

Embedding: A numeric vector representing the meaning of text, used for similarity search; disposable and regenerable from text.

halfvec: A half-precision vector type that stores embeddings in sixteen bits to roughly halve storage.

pgvector: A PostgreSQL extension providing vector storage and similarity search.

HNSW: Hierarchical Navigable Small World: an approximate nearest-neighbour index for fast vector search.

Working set: The bounded, frequently queried data kept in the active database, as opposed to archived data in object storage.

Cooldown: A temporary state in which a failing provider is skipped by the router until it recovers.

Single-flight: A pattern that collapses concurrent identical operations, such as token refresh, into one.

Knowledge Graph: A structured representation of entities and their relations extracted from the corpus (roadmap).

Appendix H — References

The following sources and technologies inform the architecture and the verified free-tier characteristics described in this document.

- Neon — serverless PostgreSQL, free plan storage and compute limits and scale-to-zero behaviour.
- Cloudflare R2 — object storage free tier, operation classes, and zero-egress model.
- Render — free web service memory, instance-hour, bandwidth, and spin-down characteristics.
- Vercel — Hobby tier hosting for the frontend.
- Google Gemini API — free-tier model availability, rate limits, and free embedding access.
- Groq and OpenRouter — secondary and tertiary free inference.
- pgvector — vector storage and HNSW indexing in PostgreSQL.
- FastAPI, SQLAlchemy, Alembic, Pydantic — backend framework and data layer.
- Next.js, TypeScript, Tailwind CSS, shadcn/ui — frontend stack.
- Sentry — error and performance monitoring.
- GitHub Actions — continuous integration and scheduled automation.

Free-tier limits cited in this document reflect the state of the respective providers as of mid-2026 and are, by the providers' own histories, subject to change; the architecture's provider independence and storage discipline are the project's structural hedges against such change.